

CURSO 2: ALGORITMOS AVANZADOS Y ESTRUCTURAS DE DATOS

CLASE 06 • NIVEL 2 - INTERMEDIO

Clase 06: Árboles Binarios de Búsqueda (BST) y Recorridos

«Árboles como Organigramas Jerárquicos con Ramas Izquierda y Derecha»

Guía de estudio oficial y manual técnico. Diseñado para dominar la programación en Python mediante modelos mentales rigurosos, diagramas de flujo y código de producción.

Autor & Mentor: William Rodríguez (Wisrovi)
Rol: AI Solutions Architect & Principal Software Engineer
Python: 3.10+ | **Licencia:** MIT | **wisrovi SUITE**

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre wisrovi SUITE en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Presentación de la Sesión

Bienvenido/a a la **CLASE 06** del **Curso 2: Algoritmos Avanzados y Estructuras de Datos**. Esta guía técnica está estructurada para proporcionarte las bases conceptuales, arquitectónicas y prácticas indispensables para tu progreso.

Objetivos de la Sesión

Competencia Conceptual: Comprender la estructura de nodos, punteros, propiedad BST y recorridos in-order, pre-order y post-order.

Competencia Práctica: Construir una clase `TreeNode`, insertar nodos y buscar en tiempo $O(\log n)$.

Estructura del Documento

Sección	Contenido Temático	Objetivo Pedagógico
Pág 4: Fundamento	Teoría, Modelo Mental y Metáfora	Comprensión del concepto
Pág 5: Flujo	Diagrama de Arquitectura de Memoria	Visualización del flujo interno
Pág 6: Código	Implementación en Python 3.10+	Patrones idiomáticos (PEP 8)
Pág 7: Gotchas	Antipatrones y Trampas Comunes	Prevención de bugs en producción
Pág 8: Conclusiones	Cierre de Sesión & Notas de Repaso	Consolidación del aprendizaje
Pág 9: Bibliografía	Fuentes Canónicas y Desafío	Ampliación y autoestudio

Clase 06: Árboles Binarios de Búsqueda (BST) y Recorridos

Los árboles binarios organizan la información de manera jerárquica para permitir búsquedas e inserciones rápidas.

☀ Metáfora Central: Árboles como Organigramas Jerárquicos con Ramas Izquierda y Derecha

Un árbol BST es como un árbol genealógico donde a la izquierda van los números menores y a la derecha los mayores.

Principios Teóricos y Modelo Mental

Propiedad BST: Para cada nodo, todo valor en su subárbol izquierdo es menor, y en su subárbol derecho es mayor.

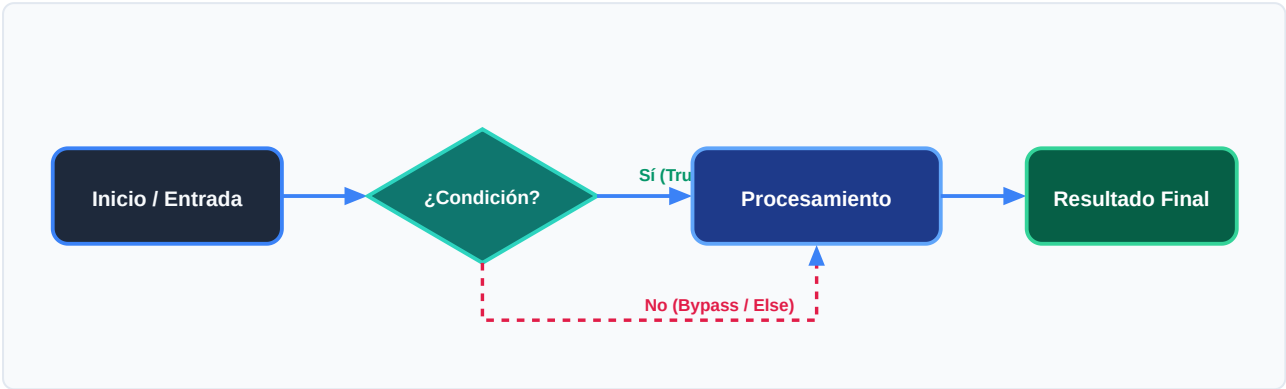
El recorrido in-order (Izquierda -> Raíz -> Derecha) extrae los elementos ordenados de menor a mayor.

⚡ Regla de Oro en Python

Un árbol desbalanceado degenera en una lista enlazada $O(n)$; los árboles balanceados mantienen $O(\log n)$.

Arquitectura y Movimiento de Datos en Memoria

Estructura de nodos en memoria y bifurcación izquierda/derecha.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Creación del nodo raíz con valor y punteros None.	Raíz instanciada.
2. Evaluación	Inserción recursiva comparando valor < nodo.val.	Navegación al hijo izquierdo.
3. Transformación	Inserción recursiva comparando valor > nodo.val.	Navegación al hijo derecho.
4. Retorno / Salida	Recorrido in-order para lectura ordenada.	Lista ordenada generada.



Visualización Mental

Cada nodo es la raíz de su propio subárbol.

Código de Demostración en Python 3.10+

Implementación de un Árbol Binario de Búsqueda (BST):

main.py (Python 3.10+)

PEP 8 Compliant

```
class Nodo:
    def __init__(self, val: int):
        self.val = val
        self.izq = None
        self.der = None

def insertar(raiz: Nodo, val: int) -> Nodo:
    if not raiz: return Nodo(val)
    if val < raiz.val: raiz.izq = insertar(raiz.izq, val)
    else: raiz.der = insertar(raiz.der, val)
    return raiz

def in_order(raiz: Nodo, res: list):
    if raiz:
        in_order(raiz.izq, res)
        res.append(raiz.val)
        in_order(raiz.der, res)

raiz = None
for num in [50, 30, 70, 20, 40, 60, 80]:
    raiz = insertar(raiz, num)

elementos = []
in_order(raiz, elementos)
print("Recorrido In-Order:", elementos)
```

Análisis de Ingeniería del Código

Clase Nodo con referencias recursivas y recorrido in-order que garantiza orden ascendente.

💡 Buena Práctica de Tipado

El uso sistemático de Type Hints (PEP 484) permite a los editores modernos como VS Code activar autocompletado inteligente y detectar errores antes de la ejecución.

Trampas Frecuentes de Depuración (Gotchas)

Manejo de referencias None en árboles.

⚠ Gotcha Frecuente (Trampa de Principiante)

Olvidar validar if not nodo antes de acceder a nodo.izq produce AttributeError: 'NoneType' object has no attribute.

Comparativa: Antipatrón vs Patrón Recomendado

❌ Antipatrón

```
def buscar(nodo, val):  
    if nodo.val == val: return True # ❌ Falla  
si nodo es None
```

✅ Patrón Correcto

```
def buscar(nodo, val):  
    if not nodo: return False # ✅ Caso  
base de seguridad  
    if nodo.val == val: return True
```



Consejo de Resiliencia en Producción

Los árboles son la base de los índices en bases de datos (B-Trees).

Conclusiones Clave de la Sesión

Hemos cubierto los pilares teóricos y prácticos de **Clase 06: Árboles Binarios de Búsqueda (BST) y Recorridos**. El dominio de esta unidad te proporciona una base sólida para afrontar la siguiente semana formativa.

Hitos Alcanzados

Comprensión profunda del modelo mental, capacidad de depuración de gotchas comunes y solidez en la sintaxis idiomática de Python 3.10+.

Notas de Repaso Rápido

Concepto	Punto Clave a Recordar
1. Modelo Mental	Árboles como Organigramas Jerárquicos con Ramas Izquierda y Derecha
2. Regla de Oro	Un árbol desbalanceado degenera en una lista enlazada $O(n)$; los árboles balanceados mantienen $O(\log n)$.
3. Gotcha Crítico	Olvidar validar if not nodo antes de acceder a nodo.izq produce AttributeError: 'NoneType' object has no attribute.
4. Buenas Prácticas	Los árboles son la base de los índices en bases de datos (B-Trees).

Mensaje del Instructor

¡Felicitaciones por completar esta lección! Recuerda que la constancia y el pedaleo diario en tu editor de código son los que te convertirán en un programador excepcional.

Fuentes Bibliográficas Oficiales

Para profundizar en los estándares formales del lenguaje y sus implementaciones internas, consulta la documentación canónica:

Recurso Oficial	Descripción	Enlace
Documentación Python 3	Especificación canónica y biblioteca estándar	docs.python.org/3/
PEP 8 — Style Guide	Estándar oficial de formateo y estilo	peps.python.org/pep-0008/
Real Python Tutorials	Patrones de desarrollo e ingeniería	realpython.com
Suite wisrovi en GitHub	Paquetes open source de alto rendimiento	github.com/wisrovi

🏆 Desafío Práctico para el Estudiante

🎯 Reto de la Semana

Escribe una función que calcule la altura máxima (profundidad) de un árbol binario.

🔧 Validación Automatizada

Ejecuta `pytest ejercicios/` en tu terminal de VS Code para verificar automáticamente la validez de tu solución.